DVA Contest 2

Comprehensive Study Notes

Python · NumPy · Pandas · Matplotlib · Seaborn

Topics Covered

1. Introduction to Python Libraries
2. NumPy — Arrays, Indexing, Boolean Masking, Operations
3. Data Loading & Cleaning
4. Data Filtering, Grouping, Merging, Pivoting
5. Descriptive Statistics & Frequency Analysis
6. Correlation Analysis
7. Outlier Detection
8. Window Functions

0 1

1 Introduction to Python Libraries **2**
1.1 Standard Imports . . . 2
1.2 Library Overview . . . 2

2 NumPy **2**
2.1 Arrays — Creation . . . 2
2.2 Indexing & Slicing . . . 3
2.3 Boolean Masking . . . 3
2.4 Vector Operations . . . 4
2.5 Numerical Operations . . . 4
2.6 Matrix Operations . . . 5

3 Pandas — Data Loading **5**
3.1 Reading Data . . . 5
3.2 Creating DataFrames Manually . . . 6

4 Data Analyzing and Cleaning **6**
4.1 Handling Missing Values (Blanks) . . . 6
4.2 Handling Duplicates . . . 7
4.3 Inconsistent Data Types . . . 8
4.4 Renaming and Dropping Columns . . . 8

5 Data Filtering **8**
5.1 Basic Filtering . . . 9
5.2 Selecting Rows and Columns . . . 9

6 Data Grouping **10**
6.1 groupby Basics . . . 10
6.2 Group by Multiple Columns . . . 10

7 Data Merging **11**
7.1 merge() — SQL-style Joins . . . 11
7.2 concat() — Stacking DataFrames . . . 12

8 Data Pivoting **12**
8.1 pivot_table() . . . 12
8.2 melt() — Wide to Long Format . . . 13

9 Descriptive Statistics **14**
9.1 Summary Statistics . . . 14

10 Frequency Analysis **15**
10.1 value_counts() and Cross-tabulation . . . 15

11 Correlation Analysis **16**
11.1 Correlation Methods . . . 16

12 Outlier Detection **17**
12.1 IQR Method 17
12.2 Z-Score Method 17
12.3 Visualising Outliers 18

13 Window Functions **18**
13.1 Rolling Window 19
13.2 Expanding Window 19
13.3 Exponentially Weighted Window (EWM) 20
13.4 shift() — Lag Features 20
13.5 rank() — Window Ranking 20

14 Quick Reference Cheat Sheet **21**

1 1

1.1 Standard Imports

Every data analysis script begins with these imports:

```
import numpy as np            # Numerical operations
import pandas as pd           # Data manipulation
import matplotlib.pyplot as plt   # Plotting
import seaborn as sns         # Statistical visualisation

# Optional style settings
sns.set_theme(style= whitegrid )
plt.rcParams['figure.figsize'] = (10, 6)
```

1.2 Library Overview

Quick Library Reference

Library	Alias	Primary Use
NumPy	`np`	Arrays, maths, linear algebra
Pandas	`pd`	DataFrames, data wrangling
Matplotlib	`plt`	Low-level plotting
Seaborn	`sns`	High-level statistical plots

2 1

2.1 Arrays — Creation

```
import numpy as np

# 1D array
a = np.array([1, 2, 3, 4, 5])

# 2D array (matrix)
b = np.array([[1, 2, 3],
              [4, 5, 6],
              [7, 8, 9]])

# Useful constructors
zeros  = np.zeros((3, 4))        # 3x4 matrix of zeros
ones   = np.ones((2, 3))         # 2x3 matrix of ones
eye    = np.eye(3)               # 3x3 identity matrix
rng    = np.arange(0, 10, 2)    # [0, 2, 4, 6, 8]
linsp  = np.linspace(0, 1, 5)   # 5 equally spaced values 0..1
```

```python
rand    = np.random.randint(1, 100, size=(3, 3))  # random ints

print(a.shape)     # (5,)
print(b.shape)     # (3, 3)
print(b.ndim)      # 2
print(b.size)      # 9
print(b.dtype)     # int64
```

2.2 Indexing & Slicing

```python
a = np.array([10, 20, 30, 40, 50])

# Basic indexing
print(a[0])       # 10   (first element)
print(a[-1])      # 50   (last element)
print(a[1:4])     # [20 30 40]  (slice)
print(a[::2])     # [10 30 50]  (every 2nd element)

# 2D indexing
b = np.array([[1,2,3],[4,5,6],[7,8,9]])
print(b[1, 2])     # 6   (row 1, col 2)
print(b[0, :])     # [1 2 3]  (entire first row)
print(b[:, 1])     # [2 5 8]  (entire second column)
print(b[0:2, 1:3])  # [[2,3],[5,6]] (submatrix)
```

Note

NumPy slices return **views**, not copies. Modifying a slice modifies the original array. Use `.copy()` to avoid this.

2.3 Boolean Masking

```python
a = np.array([5, 15, 25, 35, 45])

# Create a boolean mask
mask = a > 20
print(mask)         # [False False  True  True  True]

# Apply mask to filter
print(a[mask])      # [25 35 45]

# Shorthand (inline condition)
print(a[a % 10 == 5])    # [5 15 25 35 45]

# Compound conditions (use & | ~)
print(a[(a > 10) & (a < 40)])    # [15 25 35]
print(a[(a < 10) | (a > 40)])    # [ 5 45]
```

```python
# np.where      conditional replacement
result = np.where(a > 20, 'big', 'small')
# ['small', 'small', 'big', 'big', 'big']
```

2.4 Vector Operations

```python
x = np.array([1, 2, 3])
y = np.array([4, 5, 6])

# Element-wise arithmetic
print(x + y)      # [5 7 9]
print(x * y)      # [4 10 18]
print(x ** 2)     # [1 4 9]
print(x / y)      # [0.25 0.4  0.5]

# Scalar broadcast
print(x + 10)     # [11 12 13]
print(x * 3)      # [3 6 9]

# Universal functions (ufuncs)
print(np.sqrt(x))    # [1. 1.41 1.73]
print(np.exp(x))     # [  2.72   7.39  20.09]
print(np.log(x))     # [0.   0.69  1.10]
```

2.5 Numerical Operations

```python
a = np.array([[1, 2, 3],
              [4, 5, 6]])

# Aggregations
print(np.sum(a))           # 21
print(np.sum(a, axis=0))   # [5 7 9]  (column-wise)
print(np.sum(a, axis=1))   # [6 15]   (row-wise)

print(np.mean(a))          # 3.5
print(np.median(a))        # 3.5
print(np.std(a))           # 1.708...
print(np.var(a))           # 2.916...
print(np.min(a))           # 1
print(np.max(a))           # 6
print(np.argmin(a))        # 0  (flat index of min)
print(np.argmax(a))        # 5  (flat index of max)

# Cumulative
print(np.cumsum(a))        # [1 3 6 10 15 21]
print(np.cumprod(a))       # [1 2 6 24 120 720]
```

2.6 Matrix Operations

```python
A = np.array([[1, 2], [3, 4]])
B = np.array([[5, 6], [7, 8]])

# Matrix multiplication (dot product)
print(np.dot(A, B))       # [[19 22] [43 50]]
print(A @ B)              # same result (PEP 465)

# Transpose
print(A.T)                # [[1 3] [2 4]]

# Determinant and inverse (requires linalg)
print(np.linalg.det(A))          # -2.0
print(np.linalg.inv(A))          # [[-2.   1. ] [ 1.5 -0.5]]

# Eigenvalues & eigenvectors
vals, vecs = np.linalg.eig(A)

# Reshape
c = np.arange(1, 13)
print(c.reshape(3, 4))    # 3 rows, 4 columns
print(c.reshape(2, -1))   # 2 rows, auto columns

# Flatten / Ravel
mat = np.array([[1,2],[3,4]])
print(mat.flatten())  # [1 2 3 4]      copy
print(mat.ravel())    # [1 2 3 4]      view
```

3 1

3.1 Reading Data

```python
import pandas as pd

# ---- CSV ----
df = pd.read_csv('data.csv')
df = pd.read_csv('data.csv', sep=';')          # semicolon delimiter
df = pd.read_csv('data.csv', encoding='utf-8')
df = pd.read_csv('data.csv', index_col=0)      # first col as index
df = pd.read_csv('data.csv', nrows=100)        # load first 100 rows
df = pd.read_csv('data.csv', skiprows=2)       # skip first 2 rows
```

```python
df = pd.read_csv('data.csv', na_values=['NA','N/A','--'])  # custom NaN

# ---- Excel ----
df = pd.read_excel('data.xlsx')
df = pd.read_excel('data.xlsx', sheet_name='Sheet1')

# ---- JSON ----
df = pd.read_json('data.json')

# ---- SQL ----
import sqlite3
conn = sqlite3.connect('database.db')
df = pd.read_sql('SELECT * FROM table_name', conn)

# ---- Quick Inspection ----
print(df.head())            # first 5 rows
print(df.tail())            # last 5 rows
print(df.shape)             # (rows, columns)
print(df.columns.tolist()) # column names
print(df.dtypes)            # data types per column
print(df.info())            # full summary
```

3.2 Creating DataFrames Manually

```python
# From dictionary
df = pd.DataFrame({
    'Name':   ['Alice', 'Bob', 'Charlie', 'David', 'Eva'],
    'Age':    [25, 30, 35, 28, 22],
    'Salary': [50000, 60000, 70000, 55000, 48000],
    'Dept':   ['HR', 'IT', 'IT', 'HR', 'Finance'],
    'Score':  [85.5, 90.0, 78.3, 92.1, 88.6]
})

print(df)
#       Name  Age  Salary     Dept  Score
# 0    Alice   25   50000       HR   85.5
# 1      Bob   30   60000       IT   90.0
# 2  Charlie   35   70000       IT   78.3
# 3    David   28   55000       HR   92.1
# 4      Eva   22   48000  Finance   88.6
```

4 1

4.1 Handling Missing Values (Blanks)

```python
# Detect missing values
print(df.isnull())              # Boolean DataFrame
print(df.isnull().sum())        # Count nulls per column
print(df.isnull().sum().sum()) # Total null count
print(df.notnull().sum())       # Count non-null per column

# Rows with ANY missing value
print(df[df.isnull().any(axis=1)])

# Drop rows/columns with missing values
df_clean = df.dropna()                    # drop any row with NaN
df_clean = df.dropna(axis=1)              # drop any column with NaN
df_clean = df.dropna(subset=['Age'])      # drop only where Age is NaN
df_clean = df.dropna(thresh=3)            # keep rows with at least 3 non-NaN

# Fill missing values
df['Age'].fillna(df['Age'].mean(), inplace=True)     # fill with mean
df['Dept'].fillna('Unknown', inplace=True)           # fill with constant
df.fillna(method='ffill', inplace=True)              # forward fill
df.fillna(method='bfill', inplace=True)              # backward fill
df.fillna(df.mean(), inplace=True)                   # fill all numeric with mean
```

4.2 Handling Duplicates

```python
# Check for duplicates
print(df.duplicated())                    # Boolean Series
print(df.duplicated().sum())              # Count of duplicates
print(df[df.duplicated(keep=False)])      # Show all duplicated rows

# Duplicate on specific columns
print(df.duplicated(subset=['Name', 'Dept']))

# Remove duplicates
df_clean = df.drop_duplicates()                      # keep first occurrence
df_clean = df.drop_duplicates(keep='last')           # keep last
df_clean = df.drop_duplicates(subset=['Name'])       # based on column
df.drop_duplicates(inplace=True)                     # in-place
```

4.3 Inconsistent Data Types

```python
# Check data types
print(df.dtypes)

# Convert types
df['Age']    = df['Age'].astype(int)
df['Salary'] = df['Salary'].astype(float)
df['Name']   = df['Name'].astype(str)

# Convert to datetime
df['Date'] = pd.to_datetime(df['Date'])
df['Date'] = pd.to_datetime(df['Date'], format='%d-%m-%Y')

# Convert to numeric (coerce errors to NaN)
df['Score'] = pd.to_numeric(df['Score'], errors='coerce')

# String cleaning (inconsistent text)
df['Name'] = df['Name'].str.strip()          # remove whitespace
df['Name'] = df['Name'].str.upper()          # uppercase
df['Name'] = df['Name'].str.lower()          # lowercase
df['Dept'] = df['Dept'].str.replace('  ', ' ')  # fix double spaces
df['Name'] = df['Name'].str.title()          # Title Case
```

4.4 Renaming and Dropping Columns

```python
# Rename columns
df.rename(columns={'Name': 'Employee', 'Dept': 'Department'}, inplace=True)

# Rename all columns at once
df.columns = ['employee', 'age', 'salary', 'department', 'score']

# Drop columns
df.drop(columns=['score'], inplace=True)
df.drop(columns=['age', 'salary'], inplace=True)

# Drop rows by index
df.drop(index=[0, 2], inplace=True)
df.reset_index(drop=True, inplace=True)   # reset index after dropping
```

5 1

5.1 Basic Filtering

```python
df = pd.DataFrame({
    'Name':   ['Alice', 'Bob', 'Charlie', 'David', 'Eva'],
    'Age':    [25, 30, 35, 28, 22],
    'Salary': [50000, 60000, 70000, 55000, 48000],
    'Dept':   ['HR', 'IT', 'IT', 'HR', 'Finance'],
})

# Single condition
it_staff = df[df['Dept'] == 'IT']
young    = df[df['Age'] < 28]

# Multiple conditions
it_senior   = df[(df['Dept'] == 'IT') & (df['Age'] > 25)]
hr_or_fin   = df[(df['Dept'] == 'HR') | (df['Dept'] == 'Finance')]
not_hr      = df[df['Dept'] != 'HR']

# isin     filter multiple values
tech_depts  = df[df['Dept'].isin(['IT', 'Finance'])]
exclude     = df[~df['Dept'].isin(['HR'])]

# String filtering
alice_team  = df[df['Name'].str.startswith('A')]
c_emp       = df[df['Name'].str.contains('a', case=False)]

# Range filtering
mid_age     = df[df['Age'].between(25, 30)]     # inclusive
high_sal    = df[df['Salary'] >= 55000]

# Using .query()     readable syntax
result = df.query( Age > 25 and Dept == 'IT' )
result = df.query( Salary > 50000 or Dept == 'HR' )
```

5.2 Selecting Rows and Columns

```python
# Select columns
print(df['Name'])                # Single column -> Series
print(df[['Name', 'Salary']]) # Multiple columns -> DataFrame

# loc     label based (inclusive on both ends)
print(df.loc[0])               # row 0
print(df.loc[0:2])             # rows 0,1,2
print(df.loc[0, 'Name'])       # specific cell
print(df.loc[0:2, 'Name':'Dept'])  # row-col slice

# iloc     integer-position based
print(df.iloc[0])              # first row
```

```python
print(df.iloc[-1])              # last row
print(df.iloc[0:3])             # rows 0,1,2
print(df.iloc[1, 2])            # row 1, col 2
print(df.iloc[:, 0:2])          # all rows, cols 0-1
```

6 1

6.1 groupby Basics

```python
df = pd.DataFrame({
    'Name':   ['Alice', 'Bob', 'Charlie', 'David', 'Eva'],
    'Age':    [25, 30, 35, 28, 22],
    'Salary': [50000, 60000, 70000, 55000, 48000],
    'Dept':   ['HR', 'IT', 'IT', 'HR', 'Finance'],
    'Score':  [85.5, 90.0, 78.3, 92.1, 88.6]
})

# Group by one column
grouped = df.groupby('Dept')

# Aggregation functions
print(grouped['Salary'].mean())      # mean salary per dept
print(grouped['Salary'].sum())       # total salary per dept
print(grouped['Salary'].max())       # max salary per dept
print(grouped['Salary'].min())       # min salary per dept
print(grouped['Salary'].count())     # count per dept
print(grouped['Age'].std())          # std dev of age per dept

# Multiple aggregations on one column
print(grouped['Salary'].agg(['mean', 'min', 'max', 'sum']))

# Different aggregations per column
result = grouped.agg({'Salary': 'mean', 'Score': 'max', 'Age':
    'min'})
print(result)

# Named aggregation (pandas >= 0.25)
result = grouped.agg(
    avg_salary=('Salary', 'mean'),
    max_score=('Score', 'max'),
    count=('Name', 'count')
)
print(result)
```

6.2 Group by Multiple Columns

```python
# Group by multiple keys
multi = df.groupby(['Dept', 'Age'])['Salary'].mean()
print(multi)

# Reset index to flatten
multi_df = multi.reset_index()
print(multi_df)

# transform     adds group stat back to original DataFrame
df['dept_avg_salary'] = df.groupby('Dept')['Salary'].transform('mean')
df['salary_rank'] = df.groupby('Dept')['Salary'].transform('rank')

# filter     keep groups satisfying a condition
big_dept = df.groupby('Dept').filter(lambda x: len(x) >= 2)
print(big_dept)
```

7 1

7.1 merge() — SQL-style Joins

```python
employees = pd.DataFrame({
    'emp_id': [1, 2, 3, 4],
    'Name':   ['Alice', 'Bob', 'Charlie', 'David'],
    'dept_id': [10, 20, 20, 30]
})

departments = pd.DataFrame({
    'dept_id':   [10, 20, 40],
    'dept_name': ['HR', 'IT', 'Marketing']
})

# INNER JOIN     only matching rows in both
inner = pd.merge(employees, departments, on='dept_id', how='inner')

# LEFT JOIN     all rows from left, matched from right
left  = pd.merge(employees, departments, on='dept_id', how='left')

# RIGHT JOIN     all rows from right, matched from left
right = pd.merge(employees, departments, on='dept_id', how='right')

# OUTER JOIN     all rows from both
```

```
outer = pd.merge(employees, departments, on='dept_id', how='outer')

# Join on different column names
result = pd.merge(employees, departments,
                  left_on='dept_id', right_on='dept_id')

# Merge with suffixes (duplicate column names)
result = pd.merge(employees, departments, on='dept_id',
                  suffixes=('_emp', '_dept'))
```

Join Types Summary

how=	Result	SQL Equivalent
'inner'	Only matching rows	INNER JOIN
'left'	All left + matched right	LEFT JOIN
'right'	All right + matched left	RIGHT JOIN
'outer'	All rows from both	FULL OUTER JOIN

7.2 concat() — Stacking DataFrames

```
df1 = pd.DataFrame({'Name': ['Alice', 'Bob'],   'Score': [85, 90]})
df2 = pd.DataFrame({'Name': ['Charlie', 'David'],'Score': [78, 92]})

# Stack rows (vertical)
combined = pd.concat([df1, df2], ignore_index=True)

# Stack columns (horizontal)
df3 = pd.DataFrame({'Grade': ['A', 'A+', 'B', 'A']})
combined = pd.concat([combined, df3], axis=1)

# Stack and add a key to identify source
combined = pd.concat([df1, df2], keys=['Batch1', 'Batch2'])
```

8 1

8.1 pivot_table()

```
sales = pd.DataFrame({
    'Month':    ['Jan','Jan','Feb','Feb','Mar','Mar'],
    'Product':  ['A','B','A','B','A','B'],
    'Sales':    [100, 200, 150, 250, 120, 230],
```

```
    'Units':    [10, 20, 15, 25, 12, 23]
})

# Basic pivot table
pivot = pd.pivot_table(
    sales,
    values='Sales',
    index='Month',          # rows
    columns='Product',      # columns
    aggfunc='sum',          # aggregation
    fill_value=0            # fill NaN with 0
)
print(pivot)

# Multiple values
pivot2 = pd.pivot_table(
    sales,
    values=['Sales', 'Units'],
    index='Month',
    columns='Product',
    aggfunc={'Sales': 'sum', 'Units': 'mean'}
)

# Add margins (totals)
pivot3 = pd.pivot_table(
    sales, values='Sales',
    index='Month', columns='Product',
    aggfunc='sum', margins=True, margins_name='Total'
)
```

8.2 melt() — Wide to Long Format

```
wide_df = pd.DataFrame({
    'Name':  ['Alice', 'Bob'],
    'Math':  [85, 90],
    'Sci':   [78, 88],
    'Eng':   [92, 85]
})

# Wide -> Long
long_df = pd.melt(
    wide_df,
    id_vars=['Name'],             # columns to keep as-is
    value_vars=['Math','Sci','Eng'],  # columns to melt
    var_name='Subject',           # new column name for variable
    value_name='Marks'            # new column name for values
)
print(long_df)
#      Name Subject  Marks
# 0  Alice    Math     85
```

```
# 1     Bob     Math      90
# 2   Alice      Sci      78 ...
```

9 1

9.1 Summary Statistics

```
df = pd.DataFrame({
    'Name':   ['Alice', 'Bob', 'Charlie', 'David', 'Eva'],
    'Age':    [25, 30, 35, 28, 22],
    'Salary': [50000, 60000, 70000, 55000, 48000],
    'Score':  [85.5, 90.0, 78.3, 92.1, 88.6]
})

# Full summary (numeric columns)
print(df.describe())
#         Age         Salary       Score
# count    5.0        5.0          5.0
# mean    28.0    56600.0         86.9
# std      4.7     8173.6          5.2
# min     22.0    48000.0         78.3
# 25%     25.0    50000.0         85.5
# 50%     28.0    55000.0         88.6
# 75%     30.0    60000.0         90.0
# max     35.0    70000.0         92.1

# Include all dtypes
print(df.describe(include='all'))

# Individual statistics
print(df['Salary'].mean())        # arithmetic mean
print(df['Salary'].median()       # median (50th percentile)
print(df['Salary'].mode())        # most frequent value
print(df['Salary'].std())         # standard deviation (sample)
print(df['Salary'].var())         # variance
print(df['Salary'].min())         # minimum
print(df['Salary'].max())         # maximum
print(df['Salary'].range())       # max - min  (use .max() - .min())
print(df['Salary'].max() - df['Salary'].min()    # range
print(df['Salary'].quantile(0.25))    # Q1
print(df['Salary'].quantile(0.75))    # Q3
IQR = df['Salary'].quantile(0.75) - df['Salary'].quantile(0.25)
print(IQR)                            # Interquartile Range
print(df['Salary'].skew())            # skewness
print(df['Salary'].kurt())            # kurtosis
```

10 1

10.1 value_counts() and Cross-tabulation

```python
df = pd.DataFrame({
    'Name':   ['Alice', 'Bob', 'Charlie', 'David', 'Eva', 'Frank'],
    'Dept':   ['HR', 'IT', 'IT', 'HR', 'Finance', 'IT'],
    'Grade':  ['A', 'B', 'A', 'A', 'B', 'A'],
    'Score':  [85, 90, 78, 92, 88, 95]
})

# Frequency count for one column
freq = df['Dept'].value_counts()
print(freq)
# IT          3
# HR          2
# Finance     1

# Relative frequency (proportion)
rel_freq = df['Dept'].value_counts(normalize=True)
print(rel_freq)
# IT          0.500
# HR          0.333
# Finance     0.167

# Sort ascending
print(df['Dept'].value_counts(ascending=True))

# Unique values
print(df['Dept'].unique())          # array of unique values
print(df['Dept'].nunique())         # count of unique values

# Cross-tabulation (two-way frequency)
ct = pd.crosstab(df['Dept'], df['Grade'])
print(ct)
# Grade     A   B
# Dept
# Finance   1   1
# HR        2   0
# IT        2   1

# Normalised crosstab
ct_norm = pd.crosstab(df['Dept'], df['Grade'], normalize='index')

# With margins (totals)
ct_totals = pd.crosstab(df['Dept'], df['Grade'], margins=True)
```

11 1

11.1 Correlation Methods

```python
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

df = pd.DataFrame({
    'Age':    [25, 30, 35, 28, 22, 40, 45],
    'Salary': [50000, 60000, 70000, 55000, 48000, 80000, 90000],
    'Score':  [85.5, 90.0, 78.3, 92.1, 88.6, 76.0, 82.4],
    'Exp':    [2, 5, 10, 4, 1, 15, 20]
})

# Pearson correlation matrix (default, linear relationship)
corr_matrix = df.corr()
print(corr_matrix)

# Spearman correlation (monotonic, robust to outliers)
corr_spearman = df.corr(method='spearman')

# Kendall correlation (rank-based)
corr_kendall  = df.corr(method='kendall')

# Single pair correlation
print(df['Age'].corr(df['Salary']))  # Pearson r between two columns

# Visualise correlation matrix as heatmap
plt.figure(figsize=(8, 6))
sns.heatmap(
    corr_matrix,
    annot=True,          # show values
    fmt='.2f',           # 2 decimal places
    cmap='coolwarm',     # colour map
    vmin=-1, vmax=1,     # fix scale -1 to 1
    linewidths=0.5
)
plt.title('Correlation Heatmap')
plt.tight_layout()
plt.show()
```

Interpreting Correlation

Value	Interpretation
$r = +1.0$	Perfect positive linear relationship
$0.7 \leq r < 1.0$	Strong positive
$0.3 \leq r < 0.7$	Moderate positive
$0 < r < 0.3$	Weak positive
$r = 0$	No linear relationship
Negative values	Inverse (same magnitudes)

12 1

12.1 IQR Method

```python
import pandas as pd
import numpy as np

df = pd.DataFrame({
    'Salary': [50000, 60000, 70000, 55000, 48000, 500000, 45000]
})

Q1  = df['Salary'].quantile(0.25)
Q3  = df['Salary'].quantile(0.75)
IQR = Q3 - Q1

lower_fence = Q1 - 1.5 * IQR
upper_fence = Q3 + 1.5 * IQR

print(f Q1 = {Q1}, Q3 = {Q3}, IQR = {IQR} )
print(f Lower fence = {lower_fence} )
print(f Upper fence = {upper_fence} )

# Identify outliers
outliers = df[(df['Salary'] < lower_fence) | (df['Salary'] > upper_fence)]
print(outliers)

# Remove outliers
df_clean = df[(df['Salary'] >= lower_fence) & (df['Salary'] <= upper_fence)]
```

12.2 Z-Score Method

```python
from scipy import stats
import numpy as np

df = pd.DataFrame({
    'Salary': [50000, 60000, 70000, 55000, 48000, 500000, 45000]
})

# Z-score: how many std deviations from the mean
z_scores = np.abs(stats.zscore(df['Salary']))
df['z_score'] = z_scores

# Typical threshold: |z| > 3 is an outlier
outliers = df[df['z_score'] > 3]
print(outliers)

# Remove outliers
df_clean = df[df['z_score'] <= 3]
```

12.3 Visualising Outliers

```python
import matplotlib.pyplot as plt
import seaborn as sns

df = pd.DataFrame({
    'Salary': [50000, 60000, 70000, 55000, 48000, 500000, 45000]
})

fig, axes = plt.subplots(1, 2, figsize=(12, 5))

# Box plot      shows IQR and whiskers
sns.boxplot(y=df['Salary'], ax=axes[0], color='lightblue')
axes[0].set_title('Box Plot      Salary')

# Histogram
df['Salary'].hist(bins=20, ax=axes[1], color='steelblue', edgecolor='black')
axes[1].set_title('Histogram      Salary')

plt.tight_layout()
plt.show()
```

13 1

13.1 Rolling Window

```python
import pandas as pd
import matplotlib.pyplot as plt

# Time-series-like data
df = pd.DataFrame({
    'Day':    range(1, 11),
    'Sales':  [100, 120, 130, 90, 150, 200, 170, 160, 180, 210]
})
df.set_index('Day', inplace=True)

# Rolling mean (moving average)
df['rolling_mean_3'] = df['Sales'].rolling(window=3).mean()

# Rolling sum
df['rolling_sum_3'] = df['Sales'].rolling(window=3).sum()

# Rolling std
df['rolling_std_3'] = df['Sales'].rolling(window=3).std()

# Rolling max / min
df['rolling_max'] = df['Sales'].rolling(window=3).max()
df['rolling_min'] = df['Sales'].rolling(window=3).min()

# Rolling with min_periods (fill even with fewer obs)
df['rm_2'] = df['Sales'].rolling(window=3, min_periods=2).mean
    ()

print(df)

# Plot
df[['Sales', 'rolling_mean_3']].plot(
    figsize=(10, 5),
    title='Sales with 3-Day Rolling Mean'
)
plt.ylabel('Sales')
plt.show()
```

13.2 Expanding Window

```python
# Expanding mean      cumulative mean up to each point
df['cum_mean']  = df['Sales'].expanding().mean()
df['cum_sum']   = df['Sales'].expanding().sum()
df['cum_max']   = df['Sales'].expanding().max()
df['cum_min']   = df['Sales'].expanding().min()

# Equivalent shortcuts
df['cumsum']  = df['Sales'].cumsum()
```

```python
df['cummax']  = df['Sales'].cummax()
df['cummin']  = df['Sales'].cummin()

print(df[['Sales', 'cum_mean', 'cum_sum']])
```

13.3 Exponentially Weighted Window (EWM)

```python
# EWM      gives more weight to recent values
df['ewm_mean'] = df['Sales'].ewm(span=3, adjust=False).mean()

# span  : N-period span (higher = smoother)
# alpha : smoothing factor directly (0 < alpha <= 1)

df[['Sales', 'rolling_mean_3', 'ewm_mean']].plot(
    figsize=(10, 5),
    title='Rolling vs EWM'
)
plt.show()
```

13.4 shift() — Lag Features

```python
# Lag      previous value (useful for % change, diff)
df['prev_sales']  = df['Sales'].shift(1)      # 1-period lag
df['sales_diff']  = df['Sales'].diff(1)       # difference from previous
df['pct_change']  = df['Sales'].pct_change() # % change from previous

# Lead      future value (negative shift)
df['next_sales']  = df['Sales'].shift(-1)

print(df[['Sales', 'prev_sales', 'sales_diff', 'pct_change']].round(3))
```

13.5 rank() — Window Ranking

```python
df_rank = pd.DataFrame({
    'Name':   ['Alice', 'Bob', 'Charlie', 'David', 'Eva'],
    'Dept':   ['HR', 'IT', 'IT', 'HR', 'IT'],
    'Salary': [50000, 60000, 70000, 55000, 65000]
})

# Global rank
df_rank['global_rank'] = df_rank['Salary'].rank(ascending=False)

```

```python
# Rank within group (like SQL RANK() OVER PARTITION BY)
df_rank['dept_rank'] = df_rank.groupby('Dept')['Salary'].rank(
    method='dense',         # 'average','min','max','dense','first'
    ascending=False
)
print(df_rank)
```

Note

rank() methods:

- `average` — average rank for ties (default)
- `min` — lowest rank for ties
- `max` — highest rank for ties
- `dense` — no gaps in ranks (like SQL `DENSE_RANK`)
- `first` — rank in order of appearance

14 1

Most-Used DataFrame Methods

Task	Code
Load CSV	`pd.read_csv('file.csv')`
Preview data	`df.head()`, `df.tail()`, `df.info()`
Shape	`df.shape`
Null count	`df.isnull().sum()`
Drop nulls	`df.dropna()`
Fill nulls	`df.fillna(value)`
Drop duplicates	`df.drop_duplicates()`
Filter rows	`df[df['col'] > val]`
Select columns	`df[['col1','col2']]`
Group & aggregate	`df.groupby('col').agg(...)`
Merge DataFrames	`pd.merge(df1, df2, on='key')`
Pivot table	`pd.pivot_table(df, ...)`
Correlation matrix	`df.corr()`
Descriptive stats	`df.describe()`
Value counts	`df['col'].value_counts()`
Rolling mean	`df['col'].rolling(n).mean()`
Rank within group	`df.groupby('g')['col'].rank()`

NumPy Quick Reference

Task	Code
Create array	`np.array([1,2,3])`
Zeros / Ones	`np.zeros((m,n))`, `np.ones((m,n))`
Range	`np.arange(start, stop, step)`
Linspace	`np.linspace(start, stop, n)`
Reshape	`arr.reshape(m, n)`
Transpose	`arr.T`
Boolean mask	`arr[arr > value]`
Matrix multiply	`A @ B` or `np.dot(A, B)`
Mean / Std / Sum	`np.mean(arr)`, `np.std(arr)`, `np.sum(arr)`
Conditional replace	`np.where(cond, x, y)`